

TRƯỜNG ĐẠI HỌC MỞ TP. HỒ CHÍ MINH

# THIẾT KẾ HỆ THỐNG

ĐỒ ÁN NGÀNH - HK3 NĂM HỌC 2025-2026

## AI LEGAL INTELLIGENCE PLATFORM

*Nền tảng hỗ trợ tra cứu, phân tích và khai thác tri thức pháp lý*

*dành cho doanh nghiệp Việt Nam*

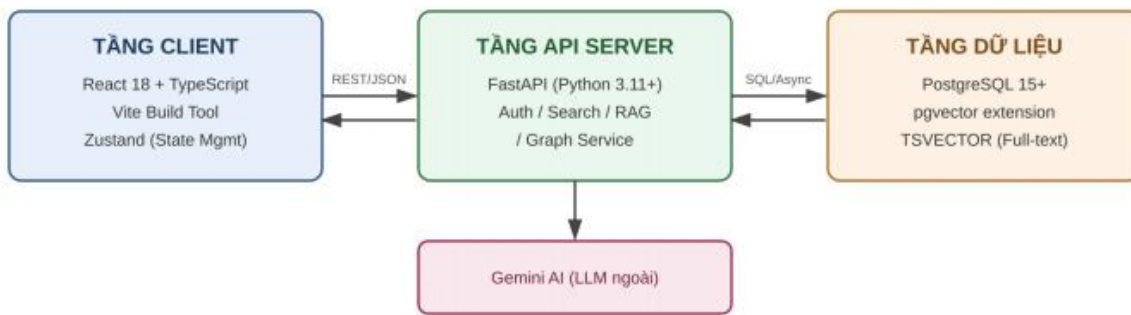
Tài liệu Thiết kế Hệ thống (System Design Document)

# CHƯƠNG 1: THIẾT KẾ KIẾN TRÚC TỔNG THỂ (ARCHITECTURE DESIGN)

Chương này mô tả cách các thành phần trong hệ thống giao tiếp với nhau, từ kiến trúc logic đến kiến trúc triển khai thực tế.

## 1.1. Kiến trúc Hệ thống (System Architecture)

Hệ thống được tổ chức theo mô hình 3 tầng (3-Tier Architecture) nhằm tách biệt rõ trách nhiệm giữa giao diện người dùng, xử lý nghiệp vụ và lưu trữ dữ liệu — giúp dễ bảo trì, kiểm thử độc lập từng tầng, và thuận tiện khi mở rộng quy mô sau này.

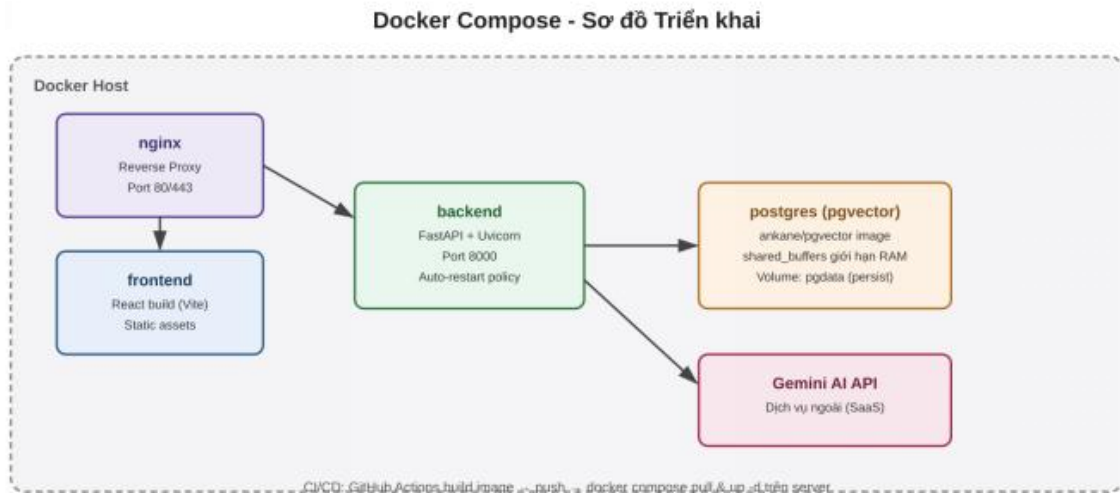


Hình 1.1 — Sơ đồ kiến trúc 3 tầng: Client / API Server / Database

| Thành phần      | Mô tả                                                                                                                                                                                                                                                                                       |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tầng Client     | React 18 + TypeScript, đóng gói bằng Vite. Quản lý trạng thái toàn cục bằng Zustand. Giao tiếp với API Server qua REST/JSON và nhận stream trả lời AI qua Server-Sent Events (SSE).                                                                                                         |
| Tầng API Server | FastAPI (Python 3.11+), xử lý bất đồng bộ (async/await). Chứa các module: Auth Router, Document Router, Search Service, RAG Service, Graph Service.                                                                                                                                         |
| Tầng Dữ liệu    | PostgreSQL 15+ với extension pgvector cho Vector Search và TSVECTOR cho Full-text Search. Một hệ quản trị CSDL duy nhất phục vụ đồng thời 3 loại truy vấn (Relational, Full-text, Vector) — giảm độ phức tạp vận hành so với việc dùng thêm một Vector DB riêng biệt (Pinecone, Milvus...). |
| Dịch vụ ngoài   | Gemini AI (LLM) được gọi từ RAG Service để sinh câu trả lời — đây là thành phần duy nhất nằm ngoài phạm vi kiểm soát trực tiếp của nhóm, cần có chiến lược chịu lỗi riêng (xem Chương 7).                                                                                                   |

## 1.2. Kiến trúc Triển khai (Deployment Architecture)

Toàn bộ hệ thống được container hóa bằng Docker Compose, phù hợp với quy mô và thời gian thực hiện đồ án (10 tuần), đồng thời vẫn đảm bảo khả năng nâng cấp lên môi trường production sau này.



Hình 1.2 — Sơ đồ triển khai Docker Compose

### Giới hạn RAM và tối ưu PostgreSQL/pgvector

Vì chạy trên hạ tầng có tài nguyên giới hạn, container postgres được cấu hình:

- Giới hạn bộ nhớ container qua `deploy.resources.limits.memory` trong `docker-compose.yml`, tránh một container chiếm hết RAM host.
- Điều chỉnh `shared_buffers` và `work_mem` của PostgreSQL phù hợp với RAM khả dụng, cân bằng giữa hiệu năng truy vấn vector (HNSW cần bộ nhớ để giữ index) và ổn định hệ thống.
- Dữ liệu được lưu trên Docker Volume (`pgdata`) để đảm bảo tính bền vững (persistence) khi container khởi động lại.
- CI/CD: GitHub Actions build image khi có thay đổi trên nhánh chính → đẩy image → deploy bằng `docker compose pull && up -d` trên server.

## 1.3. Thiết kế Component (Component Diagram)

### Frontend

- `SearchPage.tsx` — Giao diện tìm kiếm 3 chế độ hybrid/semantic/keyword với bộ lọc đa chiều (lĩnh vực, loại văn bản, hiệu lực).
- `DocumentPage.tsx` — Xem toàn văn, Timeline lịch sử và Knowledge Graph. Tích hợp Vis.js Network để trực quan hóa quan hệ văn bản.
- `AnalyticsPage.tsx` — Dashboard 5 biểu đồ (Line, Pie, Bar, Heatmap, KPI Cards) dùng thư viện Recharts.

- WorkspacePage.tsx — Quản lý Collections, Bookmarks và Ghi chú cá nhân.
- AuthPage.tsx — Đăng ký / Đăng nhập. Lưu JWT trong localStorage; tự động refresh khi hết hạn.
- State Management: Zustand store quản lý trạng thái toàn cục (user session, search filters).  
Route công khai (Guest) và route yêu cầu xác thực (User/Enterprise/Admin) được bảo vệ bằng Protected Route wrapper kiểm tra JWT trong state.
- API Layer: services/ — Tách biệt hoàn toàn logic gọi API khỏi UI component.

Backend — Các module chính

| Module            | Trách nhiệm                                                                                                                       |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Auth Router       | Đăng ký, đăng nhập, cấp/làm mới JWT, kiểm tra phân quyền (RBAC middleware).                                                       |
| Document Router   | CRUD văn bản (Admin), tra cứu chi tiết, Timeline lịch sử hiệu lực.                                                                |
| Search Service    | Điều phối Hybrid Search: gọi song song BM25 (TSVECTOR) và Semantic Search (pgvector), hợp nhất kết quả bằng RRF.                  |
| RAG Service       | Điều phối luồng Retrieval-Augmented Generation: truy vấn context liên quan, dựng prompt, gọi Gemini AI, stream kết quả về client. |
| Graph Service     | Truy vấn bằng document_relations, dựng cấu trúc node-edge phục vụ Knowledge Graph trên Frontend.                                  |
| Workspace Service | Quản lý Collection, Bookmark, Ghi chú cá nhân.                                                                                    |

1.4. Ma trận Truy vết (Traceability Matrix)

Bảng dưới đây đối chiếu các nhu cầu/Use Case đã xác định ở tài liệu Phân tích Hệ thống với thành phần thiết kế tương ứng — chứng minh Thiết kế phủ đầy đủ và không đi lệch khỏi phạm vi đã chốt:

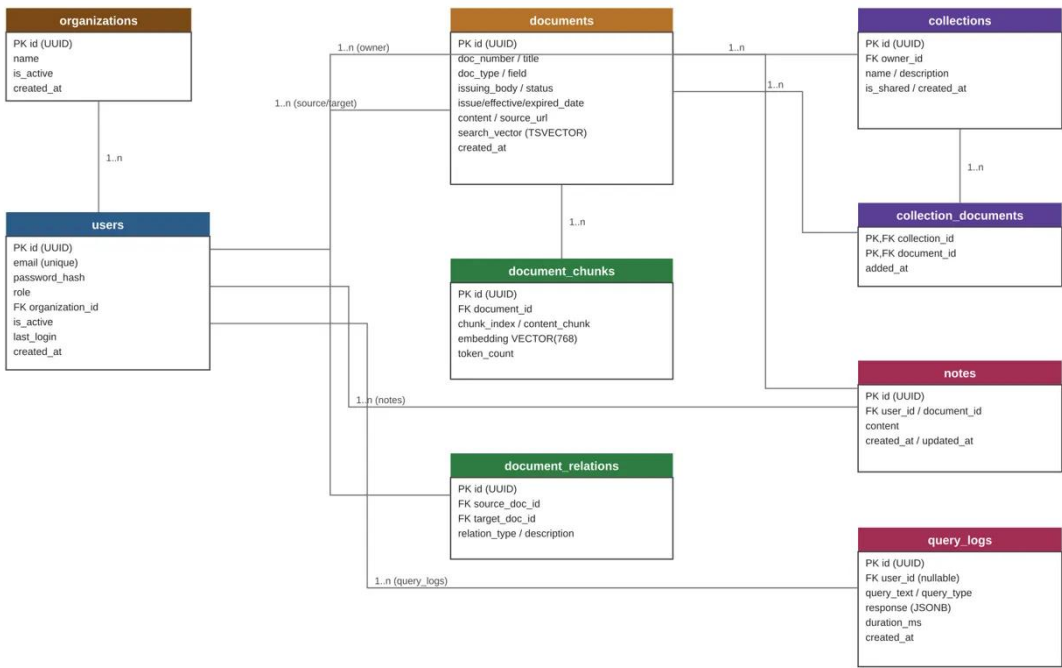
| Nhu cầu / Use Case       | Bảng Dữ liệu               | API Endpoint              | Màn hình (UI)   |
|--------------------------|----------------------------|---------------------------|-----------------|
| Tra cứu văn bản lớn      | documents, document_chunks | GET /api/v1/search        | SearchPage      |
| Hỏi đáp AI pháp lý (RAG) | documents, document_chunks | POST /api/v1/ai/chat      | Chat AI         |
| Xem Knowledge Graph      | document_relations         | GET /api/v1/graph/{id}    | Knowledge Graph |
| Tóm tắt văn bản          | documents                  | POST /api/v1/ai/summarize | DocumentPage    |
| Quản lý tài liệu cá nhân | collections, notes         | GET /api/v1/workspace     | WorkspacePage   |

|                    |           |                       |               |
|--------------------|-----------|-----------------------|---------------|
| Phân tích xu hướng | documents | GET /api/v1/analytics | AnalyticsPage |
|--------------------|-----------|-----------------------|---------------|

## CHƯƠNG 2: THIẾT KẾ DỮ LIỆU (DATABASE DESIGN)

Kiến trúc Database được thiết kế chuẩn hóa trên PostgreSQL, phục vụ đồng thời Relational query, Full-text search và Vector search thông qua extension pgvector. Chú thích: PK = Khóa chính, FK = Khóa ngoại.

### 2.1. Sơ đồ Thực thể Liên kết (ERD)



Hình 2.1 — Sơ đồ ERD giữa 9 bảng dữ liệu của hệ thống

### 2.2. Đặc tả Thiết kế Vật lý (Physical Data Model)

#### 2.2.1. Bảng documents — Văn bản pháp lý

| STT | Tên trường | Kiểu dữ liệu | Khóa | Bắt buộc | Diễn giải chi tiết         | Ví dụ        |
|-----|------------|--------------|------|----------|----------------------------|--------------|
| 1   | id         | UUID         | PK   | Có       | Định danh văn bản          | 123e4567-... |
| 2   | doc_number | VARCHAR(100) |      | Có       | Số hiệu văn bản (có Index) | 45/2019/QH14 |

|    |                |              |  |                      |                                         |                                                       |
|----|----------------|--------------|--|----------------------|-----------------------------------------|-------------------------------------------------------|
| 3  | title          | TEXT         |  | Có                   | Trích yếu / tên văn bản                 | Bộ luật Lao động 2019                                 |
| 4  | doc_type       | VARCHAR(50)  |  | Không                | Loại văn bản                            | Luật, Nghị định                                       |
| 5  | issuing_body   | VARCHAR(200) |  | Không                | Cơ quan ban hành                        | Quốc hội                                              |
| 6  | field          | VARCHAR(100) |  | Không                | Lĩnh vực pháp lý (có Index)             | Lao động, Thuế                                        |
| 7  | issue_date     | DATE         |  | Không                | Ngày ban hành                           | 2019-11-20                                            |
| 8  | effective_date | DATE         |  | Không                | Ngày có hiệu lực                        | 2021-01-01                                            |
| 9  | expired_date   | DATE         |  | Không                | Ngày hết hiệu lực (Null = còn hiệu lực) | 2025-01-01                                            |
| 10 | status         | VARCHAR(20)  |  | Có (mặc định active) | Tình trạng hiệu lực (có Index)          | active, expired                                       |
| 11 | content        | TEXT         |  | Không                | Nội dung toàn văn                       | Chương I...                                           |
| 12 | source_url     | TEXT         |  | Không                | Đường dẫn nguồn                         | <a href="https://vbpl.vn/...">https://vbpl.vn/...</a> |
| 13 | search_vector  | TSVECTOR     |  | Không                | Vector từ khóa cho Full-text search     | (tự sinh)                                             |
| 14 | created_at     | TIMESTAMPTZ  |  | Không                | Thời gian tạo bản ghi                   | 2024-03-15 10:00+07                                   |

Quan hệ: 1–n với `document_chunks`, `document_relations`, `notes`, `collection_documents`.

### 2.2.2. Bảng `document_chunks` — Dữ liệu nền cho RAG

| STT | Tên trường    | Kiểu dữ liệu | Khóa                        | Bắt buộc | Diễn giải chi tiết             | Ví dụ             |
|-----|---------------|--------------|-----------------------------|----------|--------------------------------|-------------------|
| 1   | id            | UUID         | PK                          | Có       | Định danh đoạn văn bản         | a1b2c3d4-...      |
| 2   | document_id   | UUID         | <i>FK→document<br/>s.id</i> | Có       | Văn bản gốc chứa đoạn này      | 123e4567-...      |
| 3   | chunk_index   | INTEGER      |                             | Có       | Thứ tự đoạn trong văn bản      | 1, 2, 3           |
| 4   | content_chunk | TEXT         |                             | Có       | Nội dung đoạn (256-512 tokens) | Người lao động... |

|   |             |             |  |       |                                               |                  |
|---|-------------|-------------|--|-------|-----------------------------------------------|------------------|
| 5 | embedding   | VECTOR(768) |  | Không | Vector ngữ nghĩa (bhai vietnamese-bi-encoder) | [0.12,-0.05,...] |
| 6 | token_count | INTEGER     |  | Không | Số token của đoạn                             | 256              |

Quan hệ:  $n-1$  với documents. Index HNSW trên embedding được tạo bằng migration sau khi dữ liệu đã nạp đủ, không tạo ngay lúc khởi tạo bảng.

### 2.2.3. Bảng document\_relations — Dữ liệu nền cho Knowledge Graph

| S<br>T<br>T | Tên trường    | Kiểu dữ liệu | Khóa            | Bắt buộc | Diễn giải chi tiết                              | Ví dụ                   |
|-------------|---------------|--------------|-----------------|----------|-------------------------------------------------|-------------------------|
| 1           | id            | UUID         | PK              | Có       | Định danh quan hệ                               | f9e8d7c6-...            |
| 2           | source_doc_id | UUID         | FK→documents.id | Có       | Văn bản nguồn                                   | ...                     |
| 3           | target_doc_id | UUID         | FK→documents.id | Có       | Văn bản đích                                    | ...                     |
| 4           | relation_type | VARCHAR(50)  |                 | Có       | GUIDES/AMENDS/REPLACES/REVOKES/CITES/IMPLEMENTS | AMENDS                  |
| 5           | description   | TEXT         |                 | Không    | Ghi chú thêm về quan hệ                         | Sửa đổi Điều 5, Khoản 2 |

Ràng buộc: cặp (source\_doc\_id, target\_doc\_id, relation\_type) không được lặp lại (Unique constraint).

### 2.2.4. Bảng organizations — Tổ chức/Doanh nghiệp

| STT | Tên trường | Kiểu dữ liệu | Khóa | Bắt buộc           | Diễn giải chi tiết                  | Ví dụ            |
|-----|------------|--------------|------|--------------------|-------------------------------------|------------------|
| 1   | id         | UUID         | PK   | Có                 | Định danh tổ chức                   | ...              |
| 2   | name       | VARCHAR(255) |      | Có                 | Tên tổ chức/doanh nghiệp            | Công ty TNHH ABC |
| 3   | is_active  | BOOLEAN      |      | Có (mặc định TRUE) | Trạng thái hoạt động gói Enterprise | TRUE             |
| 4   | created_at | TIMESTAMPTZ  |      | Không              | Thời gian tạo                       | 2024-03-15...    |

Ghi chú: bảng organizations được thiết kế sẵn để đảm bảo khả năng mở rộng (forward-compatible), nhưng chức năng doanh nghiệp nâng cao (Workspace chia sẻ nhóm) tạm hoãn trong phạm vi 10 tuần của đồ án — xem Mục 3.3, Mô tả đề tài. Quan hệ:  $1-n$  với users (một tổ chức có nhiều nhân sự).

### 2.2.5. Bảng users — Người dùng và phân quyền

| STT | Tên trường      | Kiểu dữ liệu | Khóa                 | Bắt buộc           | Diễn giải chi tiết          | Ví dụ                   |
|-----|-----------------|--------------|----------------------|--------------------|-----------------------------|-------------------------|
| 1   | id              | UUID         | PK                   | Có                 | Định danh người dùng        | ...                     |
| 2   | email           | VARCHAR(255) |                      | Có (Unique)        | Email đăng nhập             | user@company.com        |
| 3   | password_hash   | VARCHAR(255) |                      | Có                 | Mật khẩu mã hóa Bcrypt      | \$2b\$12\$eX...         |
| 4   | full_name       | VARCHAR(255) |                      | Không              | Tên hiển thị                | Nguyễn Văn A            |
| 5   | role            | VARCHAR(20)  |                      | Có (mặc định user) | Phân quyền RBAC             | user, enterprise, admin |
| 6   | is_active       | BOOLEAN      |                      | Có (mặc định TRUE) | Tài khoản có đang hoạt động | TRUE                    |
| 7   | organization_id | UUID         | FK→organization.s.id | Không              | Tổ chức trực thuộc          | ...                     |
| 8   | created_at      | TIMESTAMPTZ  |                      | Không              | Thời gian tạo tài khoản     | 2024-03-15...           |
| 9   | last_login      | TIMESTAMPTZ  |                      | Không              | Lần đăng nhập gần nhất      | 2024-06-01...           |

Quan hệ: n–1 với organizations; 1–n với collections, notes, query\_logs.

### 2.2.6. Bảng collections — Bộ sưu tập cá nhân/nhóm

| STT | Tên trường | Kiểu dữ liệu | Khóa | Bắt buộc | Diễn giải chi tiết   | Ví dụ |
|-----|------------|--------------|------|----------|----------------------|-------|
| 1   | id         | UUID         | PK   | Có       | Định danh bộ sưu tập | ...   |

|   |             |              |                    |                     |                            |                        |
|---|-------------|--------------|--------------------|---------------------|----------------------------|------------------------|
| 2 | name        | VARCHAR(255) |                    | Có                  | Tên bộ sưu tập             | Văn bản Thuế 2024      |
| 3 | description | TEXT         |                    | Không               | Mô tả bộ sưu tập           | Tổng hợp ND, TT về VAT |
| 4 | owner_id    | UUID         | <i>FK→users.id</i> | Có                  | Người sở hữu               | ...                    |
| 5 | is_shared   | BOOLEAN      |                    | Có (mặc định FALSE) | TRUE = chia sẻ cho tổ chức | FALSE                  |
| 6 | created_at  | TIMESTAMPTZ  |                    | Không               | Thời gian tạo              | 2024-03-15...          |

Ghi chú: trường *is\_shared* được thiết kế dự phòng cho tính năng chia sẻ nhóm (Enterprise) nhưng chưa triển khai ở phiên bản hiện tại — cùng phạm vi tạm hoãn với bảng *organizations*.

2.2.7. Bảng *collection\_documents* — Bảng trung gian *n–n*

| STT | Tên trường    | Kiểu dữ liệu | Khóa                         | Bắt buộc | Diễn giải chi tiết | Ví dụ         |
|-----|---------------|--------------|------------------------------|----------|--------------------|---------------|
| 1   | collection_id | UUID         | <i>PK,FK→collection.s.id</i> | Có       | Bộ sưu tập         | ...           |
| 2   | document_id   | UUID         | <i>PK,FK→document.s.id</i>   | Có       | Văn bản được thêm  | ...           |
| 3   | added_at      | TIMESTAMPTZ  |                              | Không    | Thời gian thêm     | 2024-03-15... |

2.2.8. Bảng *notes* — Ghi chú cá nhân trên văn bản

| STT | Tên trường  | Kiểu dữ liệu | Khóa                    | Bắt buộc | Diễn giải chi tiết   | Ví dụ                            |
|-----|-------------|--------------|-------------------------|----------|----------------------|----------------------------------|
| 1   | id          | UUID         | PK                      | Có       | Định danh ghi chú    | ...                              |
| 2   | user_id     | UUID         | <i>FK→users.id</i>      | Có       | Người tạo ghi chú    | ...                              |
| 3   | document_id | UUID         | <i>FK→document.s.id</i> | Có       | Văn bản được ghi chú | ...                              |
| 4   | content     | TEXT         |                         | Có       | Nội dung ghi chú     | Chú ý khoản 2 áp dụng từ tháng 7 |

|   |            |             |  |       |                             |               |
|---|------------|-------------|--|-------|-----------------------------|---------------|
| 5 | created_at | TIMESTAMPTZ |  | Không | Thời gian tạo               | 2024-03-15... |
| 6 | updated_at | TIMESTAMPTZ |  | Không | Thời gian cập nhật gần nhất | 2024-03-16... |

### 2.2.9. Bảng query\_logs — Log lịch sử hỏi AI

| ST T | Tên trường  | Kiểu dữ liệu | Khóa                   | Bắt buộc | Diễn giải chi tiết               | Ví dụ                             |
|------|-------------|--------------|------------------------|----------|----------------------------------|-----------------------------------|
| 1    | id          | UUID         | PK                     | Có       | Định danh log                    | ...                               |
| 2    | user_id     | UUID         | FK→users.id (SET NULL) | Không    | Người thực hiện truy vấn         | ...                               |
| 3    | query_text  | TEXT         |                        | Có       | Nội dung câu hỏi                 | Nghĩ thai sản bao lâu?            |
| 4    | query_type  | VARCHAR(20)  |                        | Không    | keyword/semantic/hybrid/ai_chat  | ai_chat                           |
| 5    | response    | JSONB        |                        | Không    | Câu trả lời AI + nguồn trích dẫn | {"text": "...", "sources": [...]} |
| 6    | duration_ms | INTEGER      |                        | Không    | Thời gian xử lý (ms)             | 2500                              |
| 7    | created_at  | TIMESTAMPTZ  |                        | Không    | Thời gian tạo log                | 2024-03-15...                     |

Phục vụ Analytics nội bộ và RAG Evaluation (đối chiếu response đã trả lời với đánh giá chất lượng sau này).

## 2.3. Thiết kế Index và Tối ưu Tìm kiếm (Crucial for AI)

### Full-text Search — search\_vector (TSVECTOR)

Trường search\_vector trên bảng documents lưu dạng biểu diễn từ khóa đã được PostgreSQL xử lý (loại bỏ từ dừng, chuẩn hóa) để phục vụ tìm kiếm BM25/ranking theo từ khóa chính xác — nhánh 'chính xác' của Hybrid Search, bổ trợ cho nhánh 'ngữ nghĩa'.

### Semantic Search — VECTOR(768) và HNSW

Trường embedding trên bảng document\_chunks lưu vector 768 chiều — đầu ra của mô hình bkaifoundation-models/vietnamese-bi-encoder. Để tìm kiếm hiệu quả trên hàng trăm nghìn vector, hệ thống dùng chỉ mục HNSW (Hierarchical Navigable Small World) thay vì IVFFlat, vì:

- HNSW không yêu cầu biết trước số lượng vector khi tạo index — phù hợp với dữ liệu tăng dần theo thời gian.
- HNSW đạt recall (độ đầy đủ kết quả đúng) cao hơn IVFFlat ở cùng mức độ trễ truy vấn.

- HNSW không cần chạy VACUUM ANALYZE sau mỗi lần insert để giữ chất lượng index — giảm chi phí vận hành.

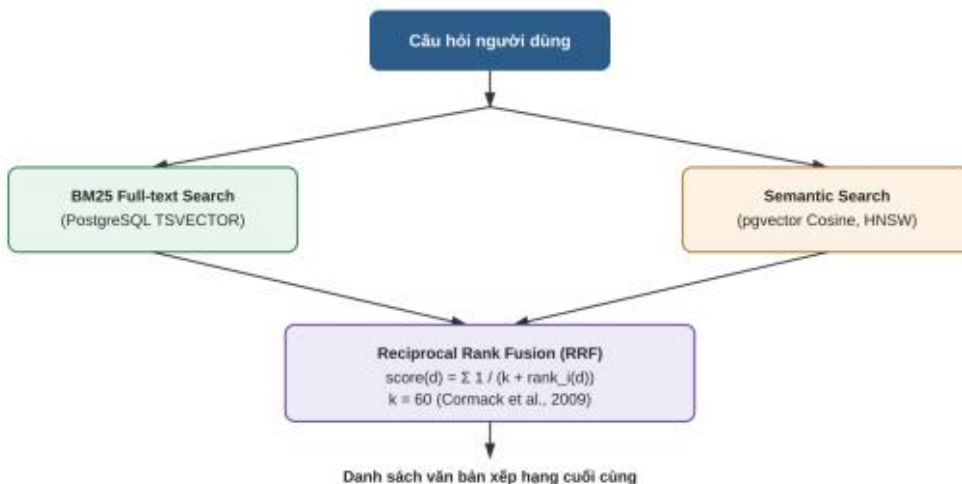
Đây là lựa chọn kỹ thuật cốt lõi quyết định tốc độ và độ chính xác của tính năng Semantic Search — điểm khác biệt kỹ thuật chính so với các hệ thống tra cứu pháp luật hiện có tại Việt Nam.

## CHƯƠNG 3: THIẾT KẾ THUẬT TOÁN & XỬ LÝ (ALGORITHM & LOGIC DESIGN)

Chương này biểu diễn luồng chạy thực tế của các tính năng lõi bằng Sequence Diagram, tập trung vào 3 luồng xử lý quan trọng nhất của hệ thống.

### 3.1. Thiết kế Luồng Hybrid Search (Search Pipeline)

Hybrid Search kết hợp BM25 (chính xác từ khóa) và Semantic Search (ngữ nghĩa) — hai phương pháp có thang điểm (scale) hoàn toàn khác nhau nên không thể cộng trực tiếp. Hệ thống dùng thuật toán Reciprocal Rank Fusion (RRF) để hợp nhất kết quả dựa trên thứ hạng (rank) thay vì điểm số thô.



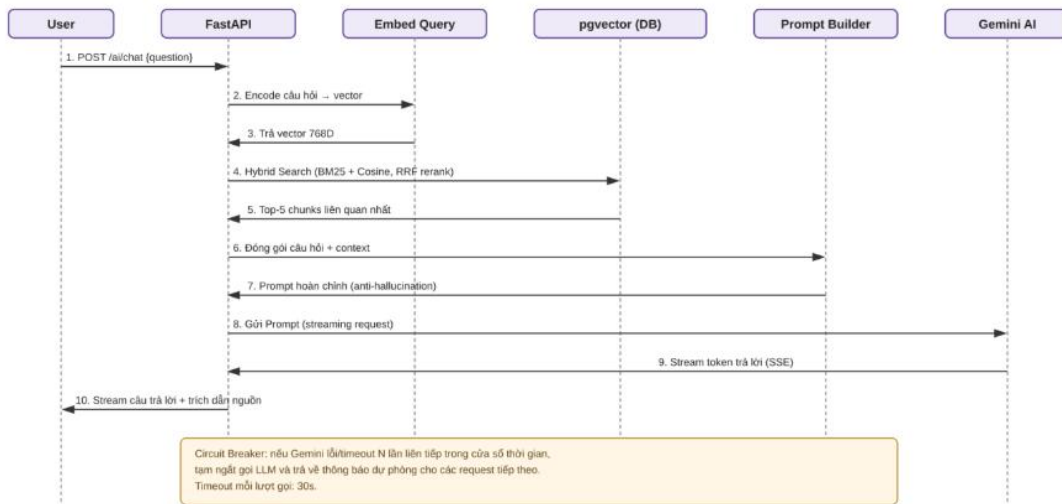
Hình 3.1 — Luồng Hybrid Search kết hợp BM25 và Semantic Search qua RRF

```
def rrf_score(rank: int, k: int = 60) -> float:
    return 1.0 / (k + rank)

def hybrid_rerank(bm25_results: list, semantic_results: list) -> list:
    scores = {}
    for rank, doc_id in enumerate(bm25_results):
        scores[doc_id] = scores.get(doc_id, 0) + rrf_score(rank)
    for rank, doc_id in enumerate(semantic_results):
        scores[doc_id] = scores.get(doc_id, 0) + rrf_score(rank)
    return sorted(scores.keys(), key=lambda x: scores[x], reverse=True)
```

k = 60 là giá trị mặc định được Cormack et al. (2009) đề xuất và được sử dụng rộng rãi trong công nghiệp cho bài toán Hybrid Search — dùng để chứng minh căn cứ học thuật của lựa chọn thuật toán.

### 3.2. Thiết kế Luồng Hối đáp AI (RAG Pipeline)



Hình 3.2 — Sequence Diagram: luồng RAG từ câu hỏi người dùng đến câu trả lời có trích dẫn

Toàn bộ luồng gồm 10 bước: mã hóa câu hỏi thành vector → Hybrid Search lấy top-5 đoạn liên quan → đóng gói (augment) thành prompt → gọi Gemini AI → stream câu trả lời kèm trích dẫn nguồn về client qua SSE.

#### System Prompt chống ảo giác (Anti-hallucination)

##### SYSTEM PROMPT:

Bạn là trợ lý pháp lý AI chuyên về pháp luật Việt Nam.

Nhiệm vụ: Trả lời câu hỏi DƯA TRÊN VÀ CHỈ DƯA TRÊN các đoạn văn bản pháp luật được cung cấp trong [CONTEXT] dưới đây.

##### NGUYÊN TẮC BẮT BUỘC:

- Nếu thông tin không có trong [CONTEXT], trả lời: "Tôi không tìm thấy đủ căn cứ pháp lý trong hệ thống. Vui lòng tham khảo luật sư."
- Mỗi luận điểm PHẢI trích dẫn: [Tên văn bản, Điều X, Khoản Y].
- KHÔNG suy diễn, KHÔNG bịa đặt thông tin ngoài [CONTEXT].
- Trả lời bằng ngôn ngữ dễ hiểu, tránh thuật ngữ pháp lý phức tạp.

[CONTEXT]: {top\_5\_retrieved\_chunks}

### 3.3. Thiết kế Luồng Xử lý Dữ liệu (ETL Pipeline)

Dữ liệu đầu vào được lấy từ các bộ dataset công khai trên HuggingFace (th1nhng0/vietnamese-legal-documents, tmquan/phapdien-moj-gov-vn...), không thực hiện crawl/scrape thủ công. Quy trình ETL gồm 4 bước:

- Extract: Tải dataset từ HuggingFace → chuẩn hóa schema, ánh xạ các trường về đúng cấu trúc bảng documents.

- Transform: Lọc theo 2 lĩnh vực trọng tâm (Lao động - BHXH, Thuế - Kế toán) → cắt nội dung thành chunk 256-512 tokens theo chiến lược sliding window có overlap 50 tokens để giữ ngữ cảnh qua ranh giới chunk.
- Embed: Sinh vector 768 chiều cho từng chunk bằng mô hình bkai-foundation-models/vietnamese-bi-encoder (đã fine-tune trên Zalo Legal Text Retrieval, Accuracy@10 = 93.59%). Xử lý theo batch để tối ưu tốc độ.
- Load: Ghi metadata vào bảng documents, vector vào document\_chunks.embedding. Build chỉ mục HNSW sau khi đã nạp đủ dữ liệu.

## CHƯƠNG 4: THIẾT KẾ API (API DESIGN)

Đặc tả các cổng giao tiếp RESTful API giữa Frontend và Backend.

### 4.1. Tiêu chuẩn Thiết kế API

- Tiền tố API: toàn bộ endpoint được đặt dưới tiền tố /api/v1/ để đảm bảo an toàn khi nâng cấp (API Versioning).
- Cấu trúc lỗi chuẩn (Standard Error Response): hệ thống thống nhất trả JSON dạng {"error": {"code": "ERROR\_CODE", "message": "Mô tả"}} thay vì text thuần, để Frontend dễ xử lý và hiển thị.
- Kiến trúc RESTful, trao đổi dữ liệu qua JSON.
- Xác thực bằng JWT Bearer Token trong header Authorization cho các endpoint yêu cầu đăng nhập.
- Endpoint AI Chat sử dụng Server-Sent Events (SSE) để stream câu trả lời theo thời gian thực thay vì chờ phản hồi trọn vẹn.
- Mã trạng thái HTTP chuẩn: 200 (thành công), 201 (tạo mới), 400 (dữ liệu không hợp lệ), 401 (chưa xác thực), 403 (không đủ quyền), 404 (không tìm thấy), 429 (vượt giới hạn tần suất), 500 (lỗi hệ thống).

### 4.2. Đặc tả các Endpoint chính

| Method | Endpoint              | Auth  | Mô tả                                                     |
|--------|-----------------------|-------|-----------------------------------------------------------|
| POST   | /api/v1/auth/register | Không | Đăng ký tài khoản, trả về JWT ngay                        |
| POST   | /api/v1/auth/login    | Không | Đăng nhập, nhận access_token (30ph) + refresh_token (7ng) |
| POST   | /api/v1/auth/refresh  | Không | Làm mới access_token bằng refresh_token                   |

|      |                                              |        |                                                  |
|------|----------------------------------------------|--------|--------------------------------------------------|
| GET  | /api/v1/auth/me                              | Bearer | Lấy thông tin tài khoản đang đăng nhập           |
| POST | /api/v1/search                               | Không  | Tìm kiếm keyword / semantic / hybrid với RRF     |
| GET  | /api/v1/search?q=&mode=                      | Không  | Tìm kiếm nhanh dạng GET                          |
| GET  | /api/v1/documents/                           | Không  | Danh sách văn bản, hỗ trợ phân trang và lọc      |
| GET  | /api/v1/documents/{id}                       | Không  | Chi tiết toàn văn và metadata một văn bản        |
| POST | /api/v1/ai/chat                              | Không* | Hỏi đáp pháp lý RAG — response dạng SSE stream   |
| POST | /api/v1/ai/summarize                         | Không* | Tóm tắt tự động một văn bản pháp lý              |
| GET  | /api/v1/graph/{doc_id}                       | Không  | Knowledge Graph nodes + edges tương thích Vis.js |
| GET  | /api/v1/analytics/dashboard                  | Không  | Dữ liệu 5 biểu đồ Dashboard theo bộ lọc          |
| GET  | /api/v1/workspace/collections                | Bearer | Danh sách Collections cá nhân                    |
| POST | /api/v1/workspace/collections                | Bearer | Tạo Collection mới                               |
| POST | /api/v1/workspace/collections/{id}/documents | Bearer | Thêm văn bản vào Collection                      |
| POST | /api/v1/workspace/notes                      | Bearer | Tạo ghi chú cá nhân trên văn bản                 |
| GET  | /api/v1/health                               | Không  | Health check — dùng cho Docker/load balancer     |

(\*) Khuyến nghị yêu cầu xác thực trong môi trường production để kiểm soát chi phí gọi LLM. Danh sách đầy đủ endpoint CRUD phụ trợ được quản lý tự động qua tài liệu OpenAPI do FastAPI tự sinh (/docs).

## CHƯƠNG 5: THIẾT KẾ GIAO DIỆN (UI/UX DESIGN)

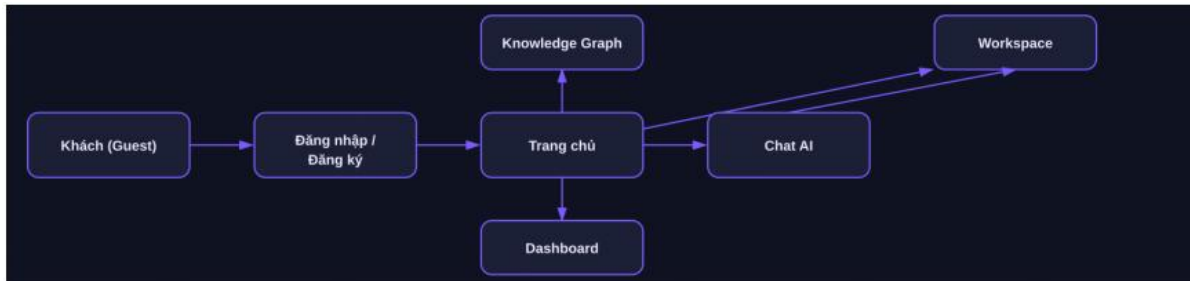
### 5.1. Phong cách Thiết kế (Design System)

Giao diện áp dụng phong cách Dark Glassmorphism (nền tối kết hợp hiệu ứng kính mờ - backdrop blur, độ trong suốt, viền phát sáng nhẹ), tạo cảm giác hiện đại và chuyên nghiệp — phù hợp định vị sản phẩm công nghệ cao cho doanh nghiệp, khác biệt với giao diện truyền thống của các hệ thống tra cứu pháp luật hiện có (thuvienphapluat.vn, vbpl.vn).

- Bảng màu nền: tông tối (navy/charcoal), điểm nhấn bằng gradient tím-xanh cho các thành phần tương tác (nút bấm, node đang chọn trong Knowledge Graph).

- Typography: font sans-serif rõ ràng, ưu tiên độ tương phản cao để đọc văn bản pháp lý dài trong thời gian dài không mỏi mắt.
- Component kính mờ dùng nhất quán cho: thẻ kết quả tìm kiếm, khung chat AI, panel chi tiết văn bản, card thống kê trên Dashboard.

## 5.2. Sơ đồ Điều hướng (Navigation Map)



Hình 5.1 — Sơ đồ điều hướng: Khách ↔ Đăng nhập ↔ Trang chủ ↔ Chat AI / Knowledge Graph / Dashboard ↔ Workspace

## 5.3. Thiết kế Màn hình (Mockups/Wireframes)

### Màn hình Tìm kiếm

- Thanh tìm kiếm trung tâm hỗ trợ cả từ khóa và câu hỏi tự nhiên.
- Bộ lọc bên trái: loại văn bản, lĩnh vực, cơ quan ban hành, khoảng thời gian, trạng thái hiệu lực.
- Danh sách kết quả dạng thẻ kính mờ, có nhãn trạng thái hiệu lực (còn/hết hiệu lực) nổi bật bằng màu.

### Màn hình Hỏi đáp AI (Chat)

- Khung chat dạng hội thoại, câu trả lời AI stream theo thời gian thực (hiệu ứng gõ chữ).
- Mỗi câu trả lời kèm danh sách nguồn trích dẫn có thể click để mở văn bản gốc.
- Ô nhập câu hỏi tiếp theo giữ ngữ cảnh hội thoại (session\_id).

### Màn hình Knowledge Graph

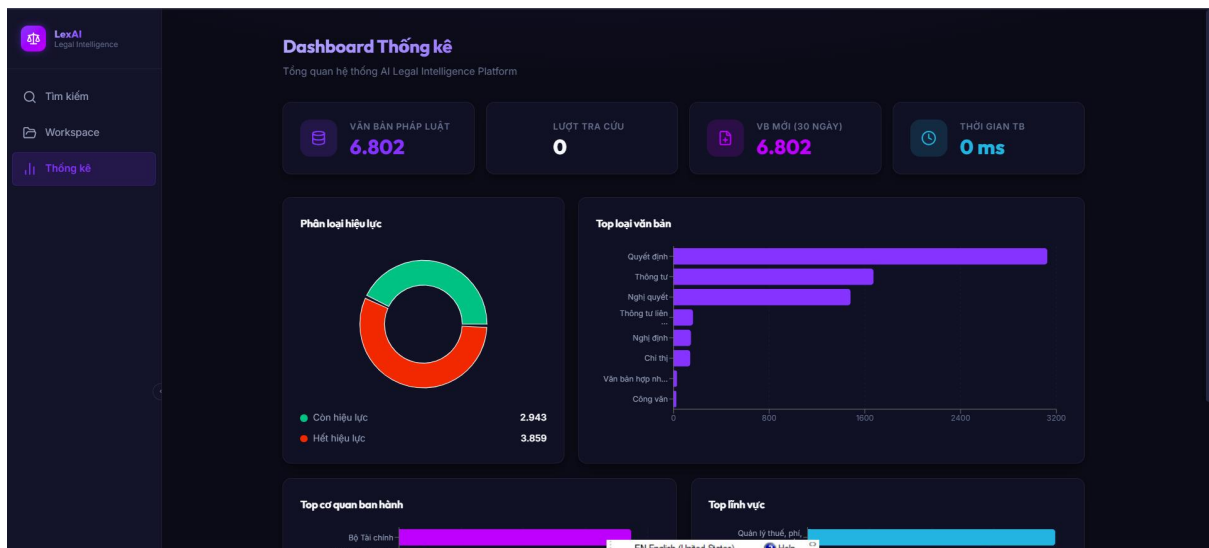
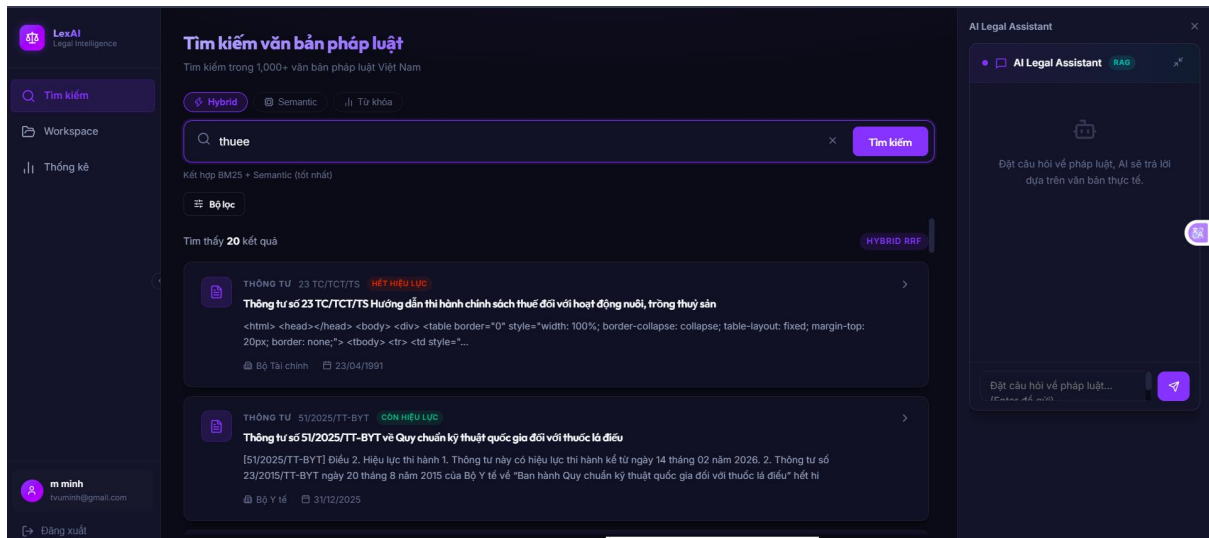
- Canvas đồ thị tương tác: văn bản trung tâm ở giữa, các văn bản liên quan tỏa ra xung quanh theo loại quan hệ (màu cạnh khác nhau cho GUIDES/AMENDS/REPLACES...).
- Click vào node hiển thị panel tóm tắt bên phải, có nút 'Xem toàn văn'.

### Màn hình Dashboard Thống kê

- Bộ lọc lĩnh vực + khoảng thời gian ở đầu trang.

- **Biểu đồ đường:** số văn bản ban hành theo năm. **Biểu đồ cột:** phân bố theo cơ quan ban hành.
- Biểu đồ tròn:** tỉ lệ còn/hết hiệu lực.

Wireframe chi tiết (Figma) sẽ được đính kèm phụ lục riêng khi hoàn thiện giao diện; phần này mô tả bố cục và luồng tương tác dự kiến cho từng màn hình.



## 5.1 Sơ đồ sơ bộ

# CHƯƠNG 6: THIẾT KẾ BẢO MẬT & AN TOÀN HỆ THỐNG

## 6.1. Bảo mật Xác thực

Hệ thống dùng chuẩn JWT (JSON Web Token) cho xác thực stateless:

- **Access Token:** thời hạn ngắn (VD: 30-60 phút), gửi kèm trong header Authorization: Bearer <token> cho mỗi request cần xác thực.

- Refresh Token: thời hạn dài hơn, dùng để cấp lại Access Token mới mà không cần đăng nhập lại, lưu ở nơi an toàn phía client (httpOnly cookie khuyến nghị).
- Middleware xác thực kiểm tra chữ ký và hạn dùng token trước khi cho phép truy cập route bảo vệ.

### 6.2. Bảo mật Dữ liệu

- Mật khẩu người dùng được mã hóa một chiều bằng Bcrypt trước khi lưu vào cột password\_hash — không bao giờ lưu plaintext.
- Kết nối giữa Client và API Server qua HTTPS trong môi trường production.
- Sử dụng ORM (SQLAlchemy) với truy vấn tham số hóa, tránh SQL Injection.

### 6.3. Phân quyền (RBAC - Role-Based Access Control)

| Vai trò         | Quyền hạn                                                                                                       |
|-----------------|-----------------------------------------------------------------------------------------------------------------|
| Guest           | Chỉ xem văn bản công khai, tìm kiếm từ khóa cơ bản, xem Dashboard tổng quan. Không dùng AI, không có Workspace. |
| User            | Toàn bộ quyền Guest + Semantic Search, AI Assistant, Knowledge Graph, Workspace cá nhân (Collection, Note).     |
| Enterprise User | Toàn bộ quyền User + Workspace chia sẻ nhóm trong cùng organization_id, xuất báo cáo.                           |
| Admin           | Toàn quyền quản trị: quản lý văn bản, quản lý người dùng, giám sát hệ thống.                                    |

*Ghi chú: vai trò Enterprise User được định nghĩa để sẵn sàng đồng bộ với bảng organizations, tuy nhiên sẽ không hoạt động trong phạm vi 10 tuần của đồ án (out-of-scope) — cùng lý do đã nêu ở Mục 2.2.4/2.2.6.*

Logic kiểm tra quyền được đặt tại tầng middleware/dependency của FastAPI (VD: Depends(require\_role("admin"))), áp dụng thống nhất cho mọi route — tránh việc kiểm tra quyền rải rác và không nhất quán trong từng handler.

### 6.4. Phòng chống Prompt Injection

Vì AI Assistant nhận input tự do từ người dùng, hệ thống áp dụng các biện pháp giảm thiểu rủi ro người dùng cố tình chèn chỉ thị nhằm ghi đè System Prompt gốc (Prompt Injection):

- System Prompt được cố định tại tầng Backend (Prompt Builder), người dùng không có cách nào truyền trực tiếp vào vị trí System role của request gửi tới Gemini AI.
- Câu hỏi người dùng chỉ được chèn vào vị trí [CONTEXT]/[QUESTION] đã quy định sẵn trong template prompt, không được nối chuỗi tự do vào phần chỉ thị hệ thống.
- Giới hạn độ dài input câu hỏi để giảm bề mặt tấn công và chi phí gọi LLM.

## CHƯƠNG 7: THIẾT KẾ XỬ LÝ LỖI & KHẢ NĂNG CHỊU LỖI (RESILIENCE DESIGN)

Vì hệ thống phụ thuộc vào dịch vụ AI bên ngoài (Gemini AI) — thành phần nằm ngoài tầm kiểm soát trực tiếp — chương này thiết kế các cơ chế đảm bảo hệ thống vẫn phản hồi hợp lý khi thành phần phụ thuộc gặp sự cố, thay vì để lỗi lan ra toàn hệ thống.

### 7.1. Circuit Breaker cho lời gọi LLM

Áp dụng mẫu thiết kế Circuit Breaker quanh lời gọi Gemini AI trong RAG Service:

- Trạng thái Closed (bình thường): mọi request được gọi thẳng tới Gemini AI.
- Khi số lần lỗi/timeout liên tiếp vượt ngưỡng trong một cửa sổ thời gian, breaker chuyển sang trạng thái Open: tạm ngắt gọi Gemini AI, trả ngay thông báo dự phòng cho người dùng (VD: "Trợ lý AI đang tạm gián đoạn, vui lòng thử lại sau ít phút hoặc dùng chức năng tìm kiếm.") thay vì để người dùng chờ timeout.
- Sau một khoảng thời gian nghỉ, breaker chuyển sang trạng thái Half-Open: cho phép một số request thử nghiệm đi qua; nếu thành công, quay lại Closed, nếu vẫn lỗi, quay lại Open.

### 7.2. Timeout và Retry Policy

- Hệ thống thiết lập thời gian chờ (timeout) cố định là 10 giây cho mỗi lần gọi LLM API — cấu hình qua hằng số `LLM_TIMEOUT_SEC = 10` trong module `rag_service.py`. Đây là ngưỡng cân bằng giữa việc cho Gemini AI đủ thời gian xử lý câu hỏi pháp lý phức tạp và trải nghiệm chờ đợi của người dùng.
- Hệ thống không thực hiện Retry tự động sau timeout vì câu hỏi pháp lý có thể không đồng bộ với ngữ cảnh nếu gọi lại — thay vào đó, Circuit Breaker kích hoạt Fallback ngay lập tức để tránh nhân đôi chi phí gọi LLM.
- Thời gian timeout 10 giây được đo từ token đầu tiên của stream (first-token latency), không phải tổng thời gian stream — đảm bảo người dùng không phải chờ quá lâu trước khi thấy phản hồi bắt đầu xuất hiện.
- Vector Search (pgvector): timeout truy vấn ngắn hơn (VD: 5 giây); nếu timeout, fallback về kết quả BM25 đơn thuần thay vì báo lỗi toàn bộ tìm kiếm.
- Embedding Service: retry tối đa 2 lần với backoff ngắn nếu lỗi tạm thời (network), sau đó trả lỗi rõ ràng cho tầng gọi.

### 7.3. Rate Limiting

Endpoint `/api/v1/ai/chat` được giới hạn tần suất gọi theo người dùng ở mức 15 request/phút/user nhằm:

- Tránh một tài khoản spam gây tổn chi phí gọi Gemini AI của toàn hệ thống.
- Đảm bảo công bằng tài nguyên giữa nhiều người dùng đồng thời trong giờ cao điểm.

Vượt giới hạn trả về mã 429 (Too Many Requests) kèm thời gian chờ gọi ý (Retry-After header).

## 7.4. Logging & Giám sát

- Mọi lượt gọi AI được ghi vào bảng query\_logs (duration\_ms, query\_type, response) — vừa phục vụ Analytics, vừa là dữ liệu để phát hiện bất thường (VD: tỉ lệ lỗi tăng đột biến).
- Log có cấu trúc (structured logging) ở tầng Backend, phân biệt rõ log nghiệp vụ và log lỗi hệ thống, thuận tiện tra cứu khi sự cố xảy ra.
- Các ngưỡng của Circuit Breaker và Rate Limiting nên được theo dõi qua log để tinh chỉnh tham số phù hợp trong quá trình vận hành thực tế.

## CHƯƠNG 8: YÊU CẦU PHI CHỨC NĂNG (NON-FUNCTIONAL REQUIREMENTS)

Các mục tiêu định lượng dưới đây là căn cứ cho những con số kỹ thuật đã đặt ra ở Chương 7 (timeout, retry, rate limit) — không phải các lựa chọn tùy ý.

- Thời gian phản hồi (Latency): truy vấn tìm kiếm thông thường (BM25/Semantic) phải hoàn thành và trả về dưới 2 giây.
- AI First-Token Latency: từ lúc người dùng gửi câu hỏi pháp lý, AI phải bắt đầu stream ký tự đầu tiên lên màn hình dưới 3 giây để đảm bảo trải nghiệm người dùng (lý do thiết lập timeout 10 giây của Circuit Breaker ở Mục 7.2).
- Sức chịu tải (Concurrency): hỗ trợ tối thiểu 50 người dùng thao tác đồng thời trên kiến trúc Docker một máy chủ (8GB RAM), sử dụng FastAPI async workers.

## KẾT LUẬN THIẾT KẾ

Tài liệu thiết kế này đã vật lý hóa hoàn toàn 6 nhu cầu cốt lõi được vạch ra ở giai đoạn phân tích (Tra cứu văn bản, Hỏi đáp AI, Tóm tắt, Dashboard, Workspace, Knowledge Graph) — đối chiếu cụ thể tại Ma trận Truy vết, Mục 1.4. Các bảng dữ liệu, API và luồng xử lý trong tài liệu giữ đúng phạm vi 10 tuần đã chốt ở Mô tả đề tài; những phần thiết kế sẵn cho khả năng mở rộng nhưng chưa triển khai (bảng organizations, trường is\_shared, vai trò Enterprise User) đều đã được chú thích rõ tại từng mục liên quan, tránh đi lệch khỏi phạm vi đã cam kết.

Tài liệu này bổ sung chi tiết kỹ thuật cho tài liệu Phân tích Hệ thống đã nộp trước đó không lặp lại nội dung Use Case, Activity Diagram và bối cảnh nghiệp vụ đã trình bày ở tài liệu đó. Mọi quyết định thiết kế cần được đối chiếu và giữ đồng bộ với code hiện hành trong quá trình triển khai.